DOCKER-01 ASSIGNMENT

Assignment Requirements

1. Set up a virtual machine using an Ubuntu image.

2. Install Docker on EC2.

3. Pull five Docker images: Nginx, Apache Tomcat, Ubuntu, Jenkins, and SonarQube.

4. Run an Nginx container and expose it on port 81.

5. Delete the Apache Tomcat image from local storage.

6. Inspect the Jenkins and SonarQube images.

7. Run a Jenkins container and execute one sample job.

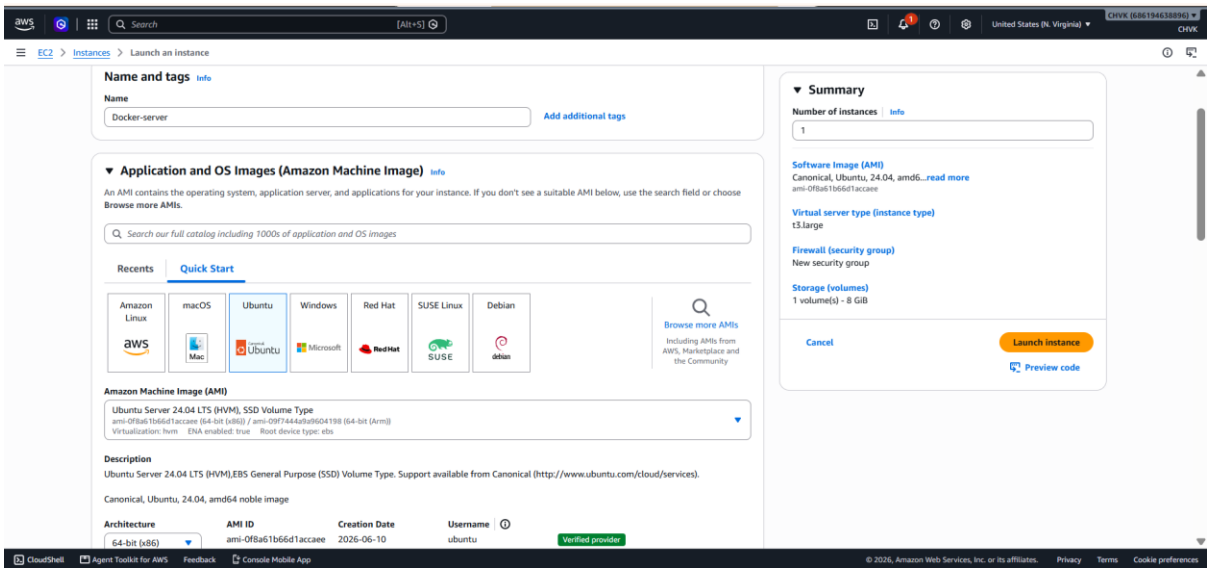
Step 1: Create an Ubuntu EC2 Instance

1. Log in to the AWS Management Console.

2. Open EC2 → Launch instance.

3. Enter the instance name: Docker-server.

4. Select Ubuntu Server 24.04 LTS.



5. Choose a suitable instance type.

6. Create or select a key pair.

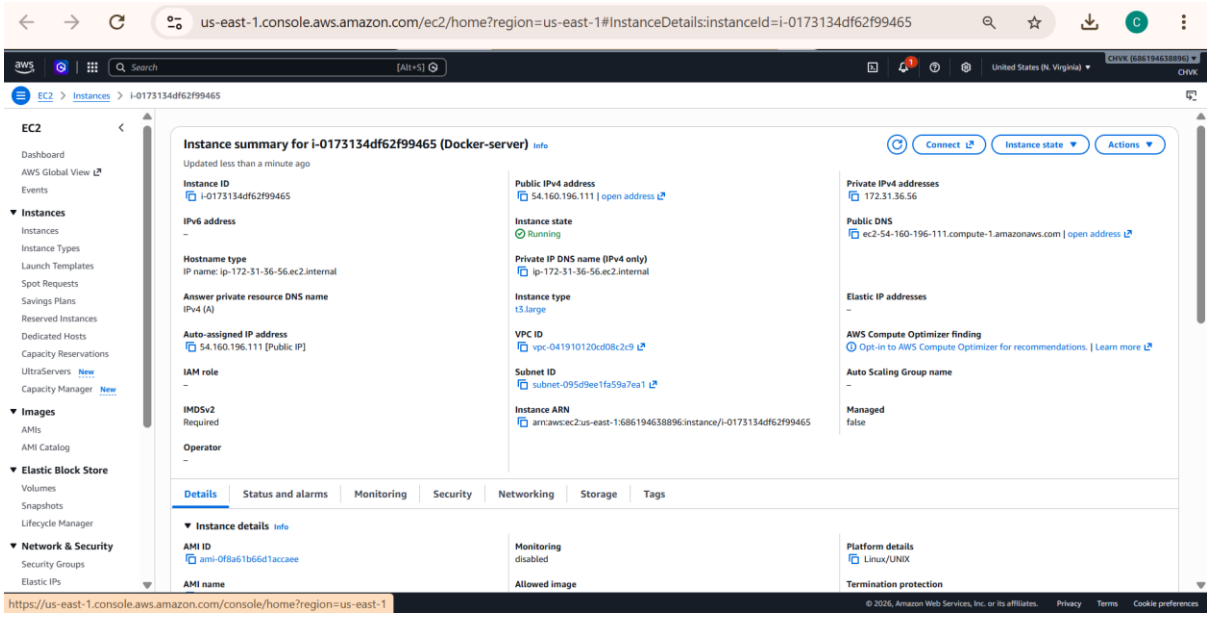
7. Configure the Security Group:

   - SSH — Port 22

   - Custom TCP — Port 81 for Nginx

- Custom TCP — Port 8080 for Jenkins

8. Click Launch instance.



9. Connect to the instance using SSH.

Result: An Ubuntu EC2 instance is ready.

```
$ ssh -i "docker-key.pem" ubuntu@ec2-54-160-196-111.compute-1.amazonaws.com
The authenticity of host 'ec2-54-160-196-111.compute-1.amazonaws.com (54.160.196.111)' can't be established.
ED25519 key fingerprint is SHA256:ppHevbhqWA3LeKl4r6Ysn50RIh0/Xb1jO7UDR8Gegpg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-54-160-196-111.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Oct 12 11:12:18 UTC 2026

  System load:  0.07              Temperature:           -273.1 C
  Usage of /:   26.6% of 6.71GB   Processes:             109
  Memory usage: 3%                Users logged in:       0
  Swap usage:   0%                IPv4 address for ens5: 172.31.36.56

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-36-56:~$ sudo su
root@ip-172-31-36-56:/home/ubuntu#
```

Step 2: Install Docker on Ubuntu

Execute the following commands one by one:

sudo apt update

sudo apt install docker.io -y

sudo systemctl start docker

sudo systemctl enable docker

Check whether Docker is installed:

docker –version

```
root@ip-172-31-36-56:/home/ubuntu# docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
root@ip-172-31-36-56:/home/ubuntu#
```

Check Docker service status:

sudo systemctl status docker

Result: Docker is installed and running on Ubuntu.

```
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /usr/lib/systemd/system/containerd.service.
Setting up ubuntu-fan (0.12.16+24.04.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service → /usr/lib/systemd/system/ubuntu-fan.service.
Setting up docker.io (29.1.3-0ubuntu3~24.04.2) ...
info: Selecting GID from range 100 to 999 ...
info: Adding group `docker' (GID 113) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-36-56:/home/ubuntu# sudo systemctl start docker
root@ip-172-31-36-56:/home/ubuntu# sudo systemctl enable docker
root@ip-172-31-36-56:/home/ubuntu# sudo ststemctl status docker
sudo: ststemctl: command not found
root@ip-172-31-36-56:/home/ubuntu# sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 11:14:14 UTC; 1min 56s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 1980 (dockerd)
      Tasks: 9
     Memory: 25.4M (peak: 25.7M)
        CPU: 502ms
     CGroup: /system.slice/docker.service
             └─1980 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Oct 12 11:14:13 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:13.551555004Z" level=info msg="Restoring containers: start."
Oct 12 11:14:13 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:13.591427806Z" level=info msg="Deleting nftables IPv4 rules" error="exit status 1"
Oct 12 11:14:13 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:13.597739753Z" level=info msg="Deleting nftables IPv6 rules" error="exit status 1"
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.012954908Z" level=info msg="Loading containers: done."
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.021651926Z" level=info msg="Docker daemon" commit="29.1.3-0ubuntu3~24.04.2" containerd-snapshotter
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.021800221Z" level=info msg="Initializing buildkit"
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.065111245Z" level=info msg="Completed buildkit initialization"
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.077395575Z" level=info msg="Daemon has completed initialization"
Oct 12 11:14:14 ip-172-31-36-56 systemd[1]: Started docker.service - Docker Application Container Engine.
Oct 12 11:14:14 ip-172-31-36-56 dockerd[1980]: time="2026-10-12T11:14:14.080443897Z" level=info msg="API listen on /run/docker.sock"
lines 1-22/22 (END)
```

Step 3: Pull Five Docker Images

A Docker image is a template used to create a container.

Execute these commands:

docker pull nginx

```
root@ip-172-31-36-56:/home/ubuntu# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
f7e1db57b58b: Pull complete
ecc510c1e359: Pull complete
04bdbc7ea6b5: Pull complete
1efc53192927: Pull complete
8020170dc65f: Pull complete
947d22046304: Pull complete
807ab930f3a0: Pull complete
a1b85ab28c90: Download complete
c329293b1d90: Download complete
Digest: sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

docker pull tomcat

```
root@ip-172-31-36-56:/home/ubuntu# docker pull tomcat
Using default tag: latest
latest: Pulling from library/tomcat
06ad70e463aa: Pull complete
4f4fb700ef54: Pull complete
a61c6f59ba30: Pull complete
4e07a0f12b2c: Pull complete
5dec61325458: Pull complete
f39241fdc590: Pull complete
130739ca07b1: Pull complete
6d27111e20e8: Pull complete
abd8fb28c7b0: Download complete
e142b487e72c: Download complete
Digest: sha256:bb7c0c78078841047e2d475acce833ef337cb0007ca7c7e4c9ff9058d9019801
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
root@ip-172-31-36-56:/home/ubuntu#
```

docker pull ubuntu

```
root@ip-172-31-36-56:/home/ubuntu# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
8f70d2bfe91a: Download complete
Digest: sha256:f144425ff09be612d6d9ad965196e9cdc23dae1f42110a8a11a3e9a8198759f7
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
root@ip-172-31-36-56:/home/ubuntu#
```

docker pull jenkins/jenkins:2.580.1-lts-jdk21

```
root@ip-172-31-36-56:/home/ubuntu# docker pull jenkins/jenkins:2.580.1-lts-jdk25
2.580.1-lts-jdk25: Pulling from jenkins/jenkins
3399524d250b: Pull complete
6eefb2f5d3e9: Pull complete
ae4aa0bd08cd: Pull complete
408777b27aa9: Pull complete
afdc5c44a377: Pull complete
5b5b28994c08: Pull complete
7ace02309cd5: Pull complete
810008851f3a: Pull complete
968c36a3b2c5: Pull complete
701ded2f9b33: Pull complete
3dd4a99993ab: Pull complete
d89089c8a5a3: Pull complete
604b733d92d1: Download complete
44136fa355b3: Download complete
Digest: sha256:42fc298cdafa2683dddde9a7ccdce4058f771b1ec56c2423dbfca82f010a455f
Status: Downloaded newer image for jenkins/jenkins:2.580.1-lts-jdk25
docker.io/jenkins/jenkins:2.580.1-lts-jdk25
root@ip-172-31-36-56:/home/ubuntu#
```

docker pull sonarqube

```
root@ip-172-31-36-56:/home/ubuntu# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
4f4fb700ef54: Pull complete
a60ef8ec184e: Pull complete
96fe3bd0148e: Pull complete
0a66c41acdcb: Pull complete
54929e68a7af: Pull complete
03217b7e6be2: Pull complete
065adb12f4f5: Pull complete
6cfeebcd7dea: Download complete
780212bdfe24: Download complete
Digest: sha256:c0f1160bccfa435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
root@ip-172-31-36-56:/home/ubuntu#
```

Check the downloaded images:

docker images

The five images are:

1. Nginx

2. Apache Tomcat

3. Ubuntu

4. Jenkins

5. SonarQube

Result: All five Docker images are downloaded to the EC2 instance.



Step 4: Run the Nginx Container on Port 81

Execute the following command:

docker run -d --name nginx-container -p 81:80 nginx

Command Explanation

- "docker run" — Creates and starts a container.

- "-d" — Runs the container in the background.

- "--name nginx-container" — Assigns a name to the container.

- "-p 81:80" — Maps EC2 port 81 to container port 80.

- "nginx" — Specifies the Docker image.

Check the running container:

docker ps

```
root@ip-172-31-36-56:/home/ubuntu# docker run -d --name nginx-container -p 81:80 nginx
b16a8f229beac96fc78e9c227e83b02fc323da21b22a5cfe29cbb72ac4182cbe
root@ip-172-31-36-56:/home/ubuntu# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                     NAMES
b16a8f229bea   nginx     "/docker-entrypoint.…"   15 seconds ago   Up 15 seconds   0.0.0.0:81->80/tcp, [::]:81->80/tcp   nginx-container
root@ip-172-31-36-56:/home/ubuntu#
```

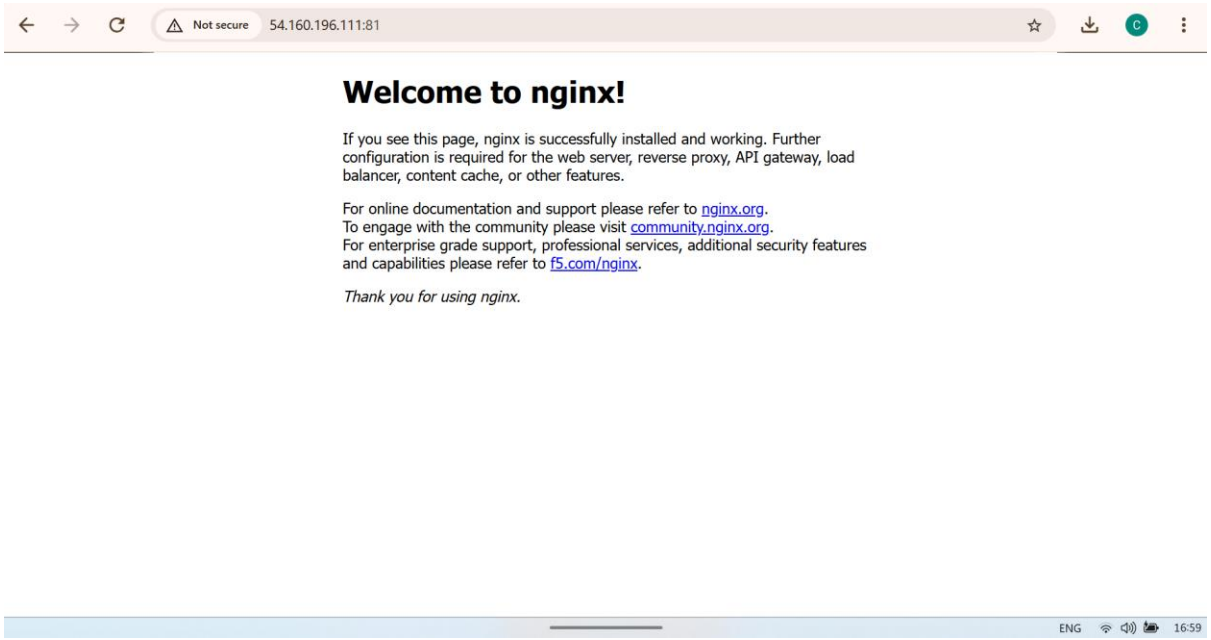
Open the browser and enter:

http://YOUR-EC2-PUBLIC-IP:81

Replace YOUR-EC2-PUBLIC-IP with your EC2 public IP address.

You should see the Nginx welcome page.

Result: The Nginx container is running and accessible through port 81.



Step 5: Delete the Apache Tomcat Image

First, list the available images:

docker images

Delete the Tomcat image:

docker rmi tomcat

Check the images again:

docker images

The Tomcat image should no longer appear in the local image list.

Note: If the image is being used by a container, Docker may require you to remove that dependency first.

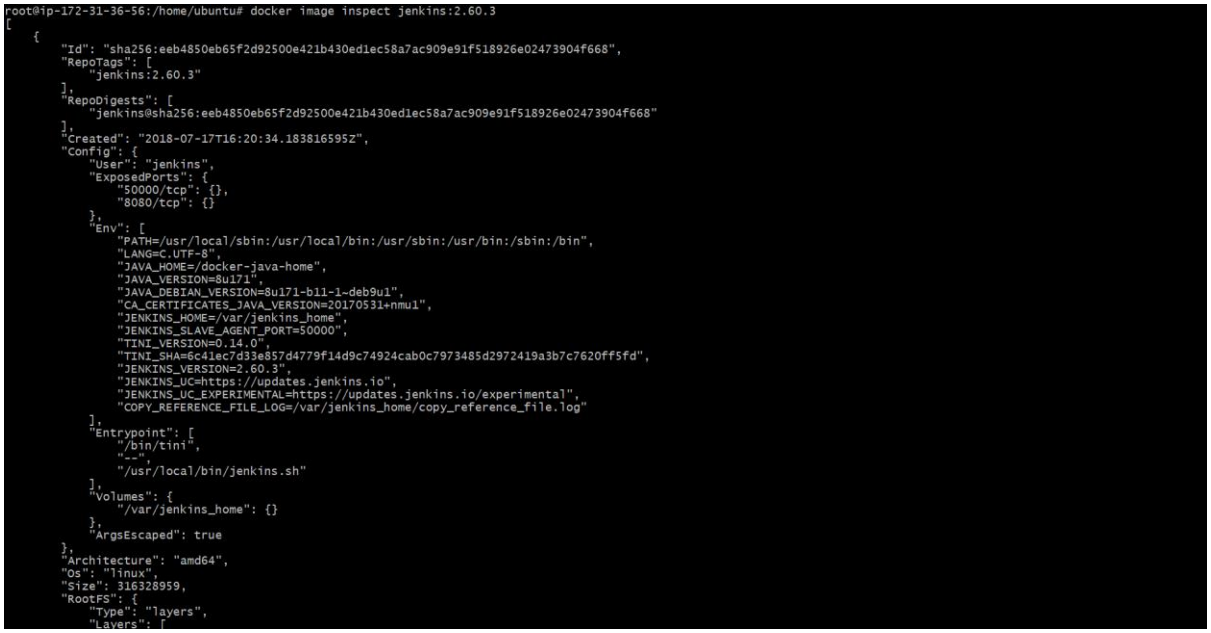
Result: The Apache Tomcat image is removed from local Docker storage.



Step 6: Inspect the Jenkins Image

Execute:

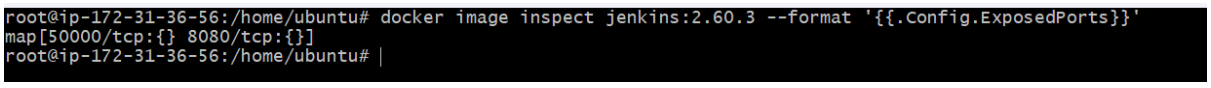
docker image inspect jenkins:2.60.3



To display only the exposed ports:

docker image inspect jenkins:2.60.3 --format '{{.Config.ExposedPorts}}'

Explanation: The "docker image inspect" command displays detailed information about an image, including its configuration and exposed ports.



Step 7: Inspect the SonarQube Image

Execute:

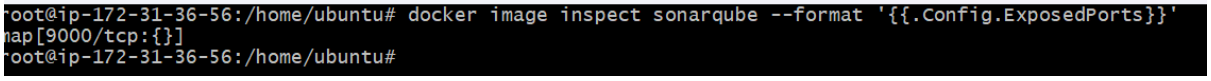
docker image inspect sonarqube



To display only the exposed ports:

docker image inspect sonarqube --format '{{.Config.ExposedPorts}}'

Explanation: This command displays the configuration and other details of the SonarQube image.



Result: Both Jenkins and SonarQube images are inspected successfully.

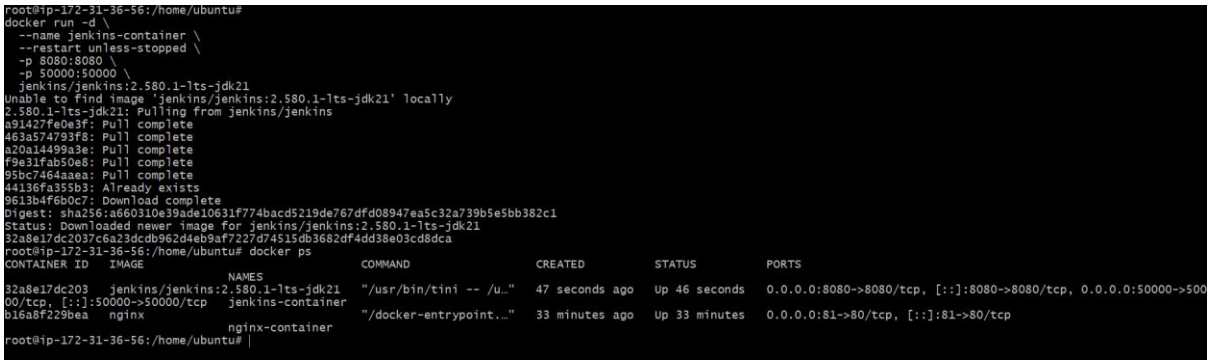
Step 8: Run the Jenkins Container

Execute the following command:

docker run -d \ --name jenkins-container \ --restart unless-stopped \ -p 8080:8080 \ -p 50000:50000 \ jenkins/jenkins:2.580.1-lts-jdk21

Check whether Jenkins is running:

docker ps



Retrieve the initial administrator password:

docker exec jenkins-container cat /var/jenkins_home/secrets/initialAdminPassword

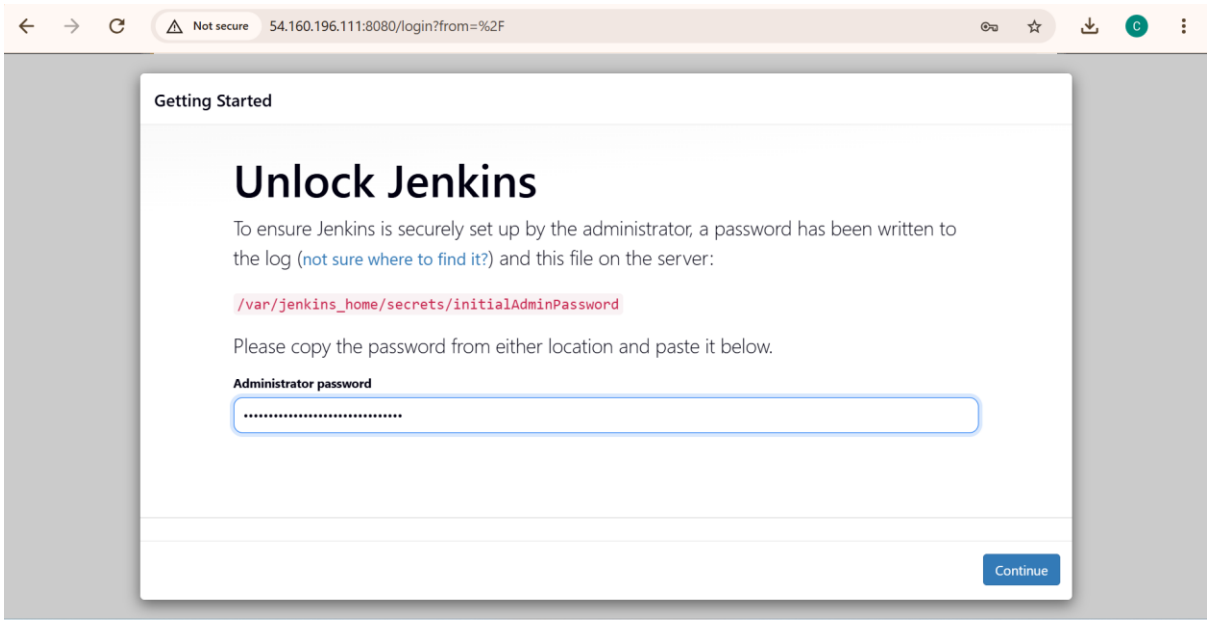
Copy the password displayed in the terminal.

Open Jenkins in your browser:

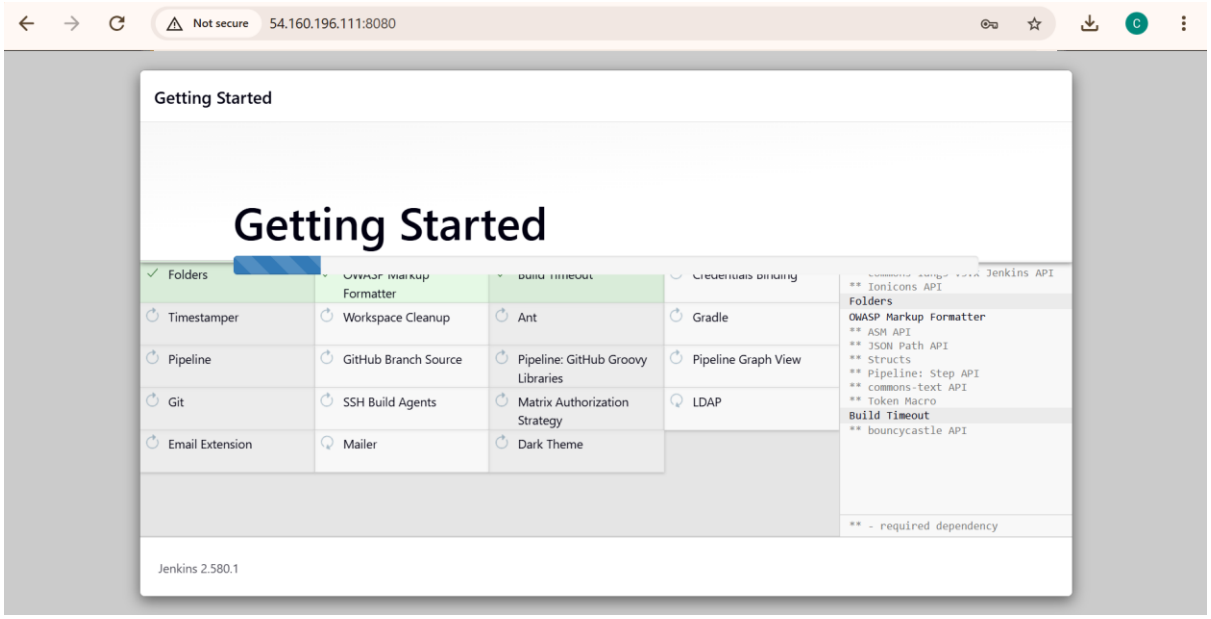
http://YOUR-EC2-PUBLIC-IP:8080

Complete Jenkins Setup

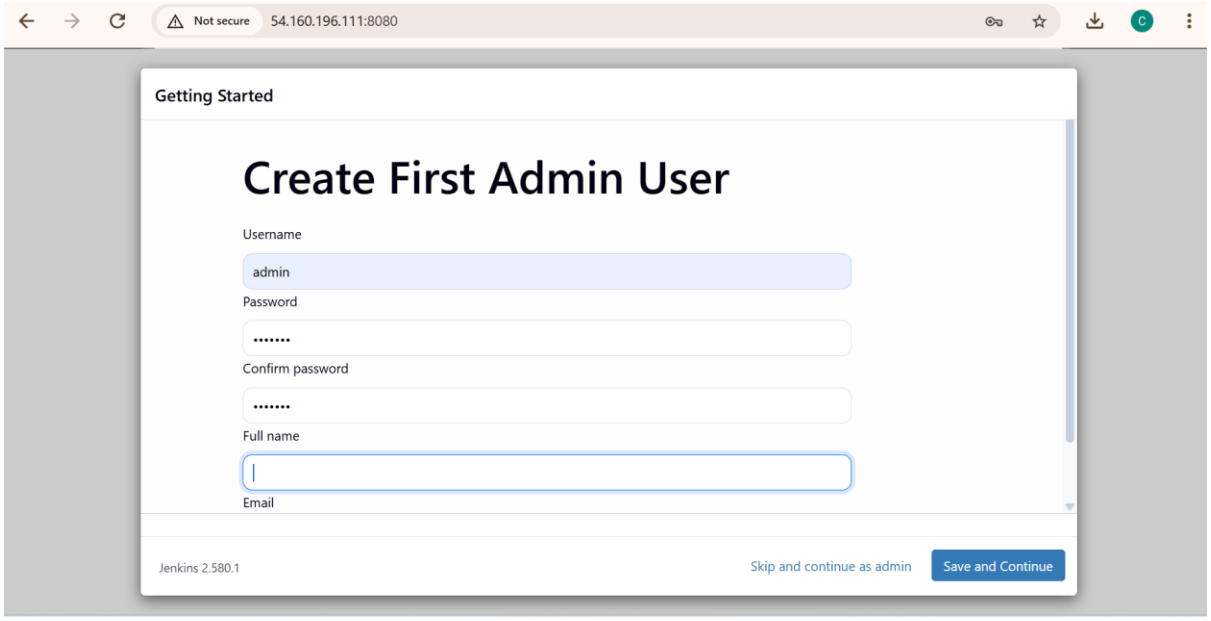
1. Enter the initial administrator password.



2. Click Install suggested plugins.



3. Create an administrator user.

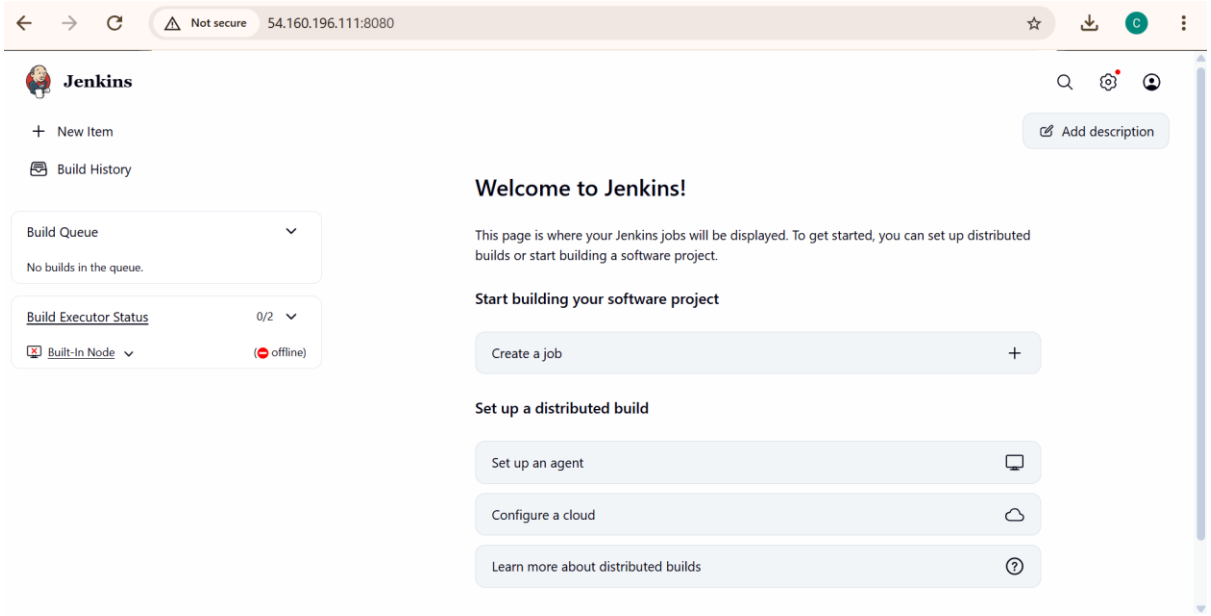


4. Finish the setup.

If Jenkins does not start, check its logs:

docker logs jenkins-container

Result: Jenkins is running inside a Docker container and accessible through port 8080.



Step 9: Create a Sample Jenkins Job

Step 1: Create a Job

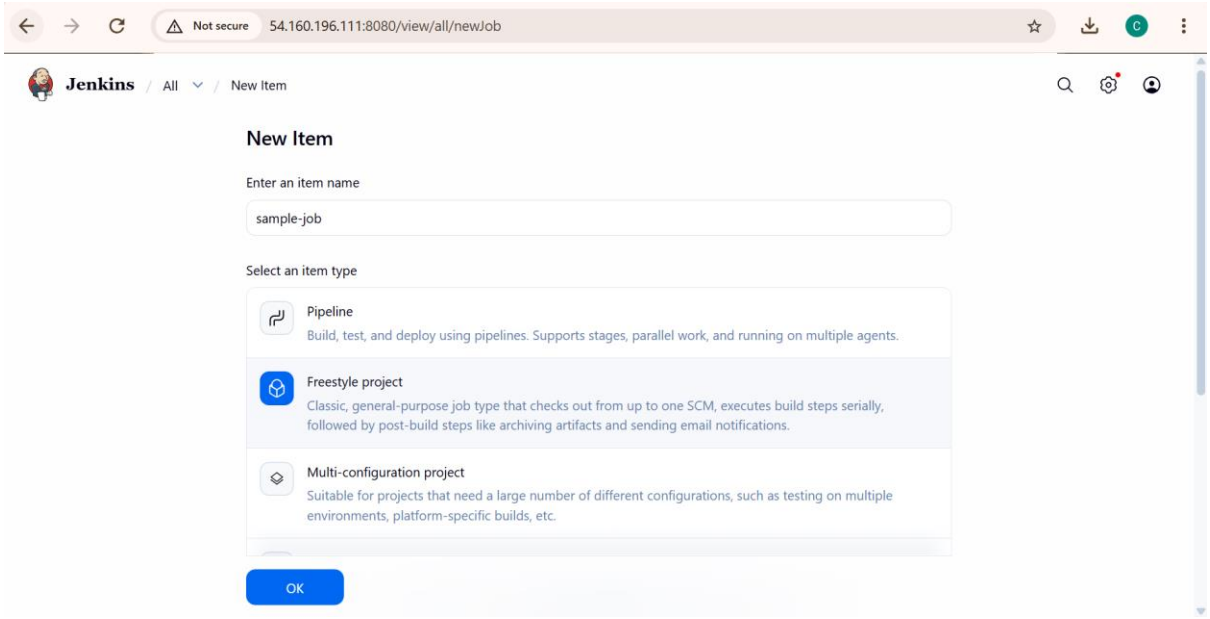
1. Open the Jenkins dashboard.

2. Click New Item.

3. Enter the job name: Sample-Job.

4. Select Freestyle project.

5. Click OK.



Step 2: Configure the Build Step

1. Scroll down to Build Steps.

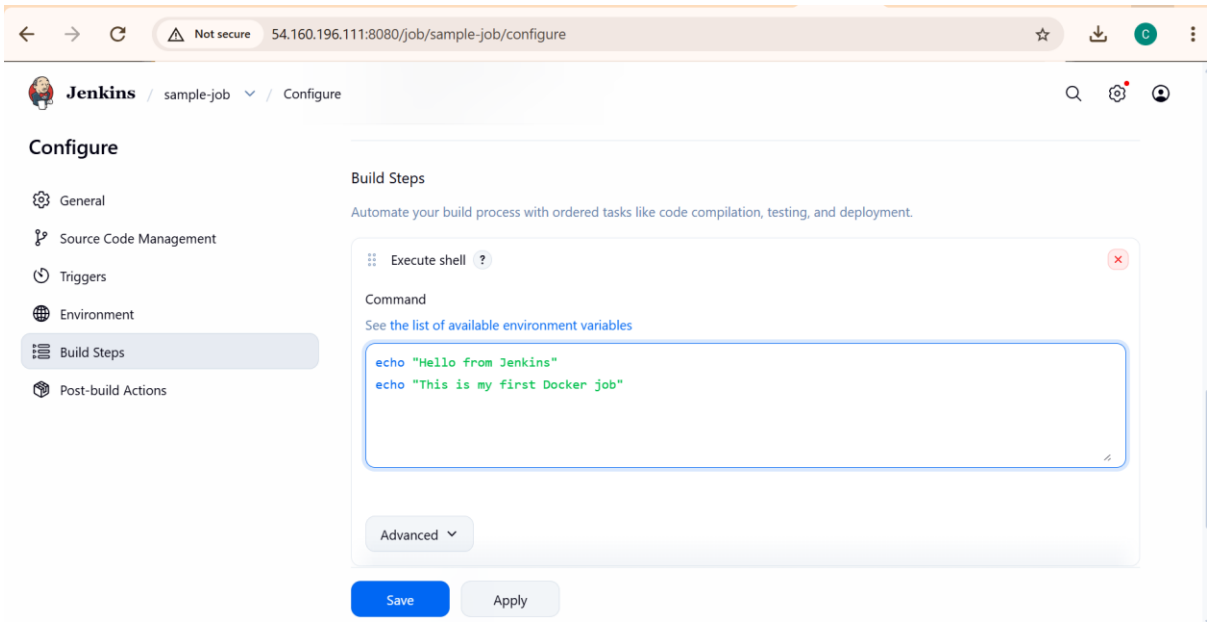
2. Click Add build step.

3. Select Execute shell.

4. Enter the following commands:

echo "Hello from Jenkins"

echo "This is my first Docker job"



Step 3: Save and Execute the Job

1. Click Save.

2. Click Build Now.

3. Open the build number from Build History.

4. Click Console Output.

Expected Output

Hello from Jenkins

This is my first Docker job

Finished: SUCCESS

Result: The sample Jenkins job executes successfully.



Important Docker Commands

- "docker images" — Lists downloaded images.

- "docker ps" — Lists running containers.

- "docker ps -a" — Lists all containers.

- "docker pull nginx" — Downloads the Nginx image.

- "docker run" — Creates and starts a container.

- "docker image inspect nginx" — Displays image details.

- "docker rmi tomcat" — Removes the local Tomcat image.

- "docker stop nginx-container" — Stops the Nginx container.

- "docker start nginx-container" — Starts the stopped Nginx container.

- "docker logs jenkins-container" — Displays Jenkins logs.

Conclusion

In this assignment, we created an Ubuntu EC2 instance, installed Docker, pulled five images, ran Nginx and Jenkins containers, deleted the Tomcat image, inspected Jenkins and SonarQube images, and executed a sample Jenkins job.